

Git & GitHub Ultimate Cheatsheet

The Definitive A-to-Z Production Reference Guide

1. SYSTEM SETUP & CONFIGURATION

```
git config --global user.name "Your Name"
```

Configure global commit author identification name.

```
git config --global user.email "you@example.com"
```

Configure global commit author identity verification email.

```
git config --global color.ui auto
```

Enable automated shell command line output colorization grids.

```
git config --global init.defaultBranch main
```

Establish your default initialization tracking branch name as main.

```
git config --list
```

List all currently active local and global metadata configurations.

2. INITIALIZING & CLONING

```
git init
```

Initialize a brand new, empty local tracking repository database.

```
git clone https://github.com/user/repo.git
```

Clone a remote cloud repository along with its history network tree.

```
git clone -b name --single-branch https://github.com/user/repo.git
```

Surgically download only a single targeted reference branch path string.

3. THE LOCAL STAGING LOOP & COMMITTING

```
git status
```

Display changes in working directory, staged indices, and untracked records.

```
git add filename.js
```

Stage specific modified properties for inclusion in the upcoming commit.

```
git add .
```

Stage all local variations, deletions, and untracked file structures across trees.

```
git add -p
```

Interactively audit and stage specific file change blocks (hunk-by-hunk execution).

```
git diff
```

Show delta mutations inside your working copy that are NOT yet staged.

```
git diff --cached
```

Show changes between staged data cache elements and last committed state layouts.

```
git commit -m "feat: design login view"
```

Lock down staged data frames inside an immutable snapshot with message notes.

```
git commit --amend
```

Replace your last local snapshot with current staged index layouts or modify logs.

4. BRANCHING & CONTEXT SWITCHING

```
git branch
```

List local tracking pointers; active operational branches append an asterisk.

```
git branch -a
```

List all local tracking structures combined with remote cloud-linked references.

```
git switch branch-name
```

Safely change directory tracking states to point to target branch timelines.

```
git switch -c feature-name
```

Surgically create a new local branch tracking path and switch HEAD to it instantly.

```
git branch -d branch-name
```

Safely drop a branch node locally (Fails if branch contains unintegrated logic logs).

```
git branch -D branch-name
```

Force kill a target local branch node regardless of upstream integration compliance.

5. INTEGRATING HISTORIES: MERGING & REBASING

```
git merge feature-branch
```

Combine changes from a feature branch into active tracking streams via merge commit.

```
git rebase main
```

Lift local feature branch commits and re-apply them on top of target branch tips.

```
git rebase --abort || git merge --abort
```

Completely terminate active history loops and restore directory layouts to pre-clash states.

```
git cherry-pick <commit-hash>
```

Isolate a single specific commit hash from any timeline and append it to active targets.

6. MANAGING REMOTES & SYNCHRONIZATION

```
git remote -v
```

Display configured cloud endpoint remote server tracking names and active URLs.

```
git remote add origin https://github.com/...
```

Bind an external repository hub tracking string mapping to a local asset name.

```
git fetch origin
```

Download metadata objects and branch pointers from cloud nodes without changing local files.

```
git pull origin main
```

Fetch latest remote tracking states and automatically merge updates into active lines.

```
git push origin branch-name
```

Upload local history snapshots into cloud repository remote storage tracks.

```
git push --force-with-lease origin feature-name
```

Safely overwrite cloud remote paths only if no third-party alterations have been pushed.

7. STASHING: STATE PRESERVATION

```
git stash
```

Shelf all uncommitted modifications onto a local temporary index memory structure.

```
git stash -u
```

Stash active modified properties along with uncommitted untracked workspace profiles.

```
git stash list
```

View all saved modification bundles sitting inside the local storage shelf.

```
git stash pop
```

Restore the top modification frame back to your working workspace and drop its memory record.

8. DISASTER RECOVERY & DIAGNOSTICS

```
git restore filename.js
```

Completely erase changes within a file, pulling back the last committed clean copy.

```
git reset --soft HEAD~1
```

Undo the last local snapshot, preserving all file changes in the staging cache.

```
git reset --hard HEAD~1
```

CRITICAL: Erase your last commit, wiping out ALL file variations permanently.

```
git reflog
```

Inspect local safety log tracking branch HEAD movements to recover missing hashes.